Related Work

Grounding documents for the failure taxonomy (spec 001) and pilot design (spec 004). Facts below were extracted from the cited sources on 2026-07-09; when something couldn't be verified from the source it is marked *(unverified)*.

1. TRAIL — Trace Reasoning and Agentic Issue Localization

Citation: Deshpande et al., *TRAIL: Trace Reasoning and Agentic Issue Localization*, arXiv:2505.08638 (Patronus AI, 2025). **Artifacts:** dataset `PatronusAI/TRAIL` (HF), code `patronus-ai/trail-benchmark`.

What it is

A benchmark of **148 human-annotated agent traces** (recorded as OpenTelemetry spans) containing **~841 errors**, drawn from GAIA (open-world information retrieval, 118 traces / 579 errors) and SWE-Bench Lite (software engineering, 31 traces / 256 errors). Avg. ~5.7 errors per trace; ~110 annotation minutes per trace. LLM judges must reproduce the human annotations: for each error, its **category**, **location** (span id), and **impact** (Low / Medium / High).

Taxonomy (3 top-level groups, ~20 fine-grained types)

Reasoning errors

Type	Definition
Hallucination (text-only)	Fabricated statements conflicting with known facts
Hallucination (tool-related)	Invented tool outputs or misrepresented tool capabilities
Poor information retrieval	Redundant / overloaded retrieval that buries the needed content
Tool output misinterpretation	Misreading correctly retrieved context or tool results
Incorrect problem identification	Misunderstanding the (sub)task at a given step
Tool selection error	Choosing an inappropriate tool for the step
Formatting errors	Malformed structured output (JSON, code)
Instruction non-compliance	Failing to follow explicit directions

System execution errors

Type	Definition
Incorrect tool definition	Inaccurate tool descriptions misleading the agent
Environment setup errors	Missing API keys, wrong permissions, broken env
API/system issues	Rate limiting (429), auth (401/403), server (500), not found (404)

Type	Definition
Resource exhaustion	Memory or resource over-allocation
Timeout issues	Infinite loops, excessive processing time

Planning & coordination errors

Type	Definition
Context handling failures	Losing episodic/semantic information across steps
Resource abuse	Repeated redundant tool calls
Goal deviation	Drifting off-task
Task orchestration errors	Multi-agent coordination failures

Key results

Best judge (Gemini-2.5-Pro): joint (category ∧ location) accuracy **0.183** on GAIA and **0.050** on SWE-Bench — i.e., ~11% combined. Trace debugging is hard for LLMs; this motivates measuring judge–human agreement (our subquestion 2) rather than assuming judge reliability.

Relevance to this project

- Source of fine-grained error types (Axis B of our taxonomy) and of the **location + impact** annotation pattern.
- The human-annotated dataset is the agreement gold standard for subquestion 2.
- Caveats for adaptation: TRAIL traces are OpenTelemetry spans from tool-calling pipelines, not terminal sessions; "task orchestration" (multi-agent) likely inapplicable to single-agent terminal runs — removal candidate, to be confirmed empirically in the pilot.

2. AgentErrorTaxonomy / AgentErrorBench / AgentDebug

Citation: Zhang et al., *Where LLM Agents Fail and How They can Learn From Failures*, arXiv:2509.25370 (2025). **Artifacts:** code + data `ulab-uiuc/AgentDebug`.

What it is

A **module-oriented** failure taxonomy: errors are attributed to the cognitive module of the agent that failed, plus a benchmark (**AgentErrorBench**: 200 annotated failure trajectories — ALFWorld 100, GAIA 50, WebShop 50) and a debugger (**AgentDebug**) that locates the root-cause step. Central empirical claim: failures **cascade** — one root-cause error propagates through subsequent decisions, so identifying the *earliest critical error* matters more than counting symptoms.

Taxonomy (5 modules, 17 error types; list from the AgentDebug repo README)

Module	Error types
Memory	hallucination; memory retrieval failure; over-simplification
Reflection	progress misjudge; outcome misinterpretation; causal misattribution; hallucination
Planning	constraint ignorance; impossible action; inefficient plan
Action	misalignment; invalid action; format error; parameter error
System	step limit; tool execution error; LLM limit; environment error

(The repo lists 18 names because "hallucination" appears under both memory and reflection; the paper counts 17 types.)

Key results

Memory and reflection errors dominate root causes; action/system errors are rarer but often immediately fatal (e.g., step-limit exhaustion). AgentDebug's root-cause feedback yields up to **26% relative** task-success improvement.

Relevance to this project

- Source of the **cognitive function axis** (Axis A): memory / reflection / planning / action / system maps well onto terminal agents (context retention, self-verification of command output, strategy, command construction, harness limits).
- Their **root-cause vs. cascade** distinction becomes our per-annotation root-cause flag and the "earliest correctable error" heuristic in the annotation guidelines.
- Their annotation is per-step per-module — matches our (cognitive function × location) requirement.

3. Terminal-Bench 2.0 + Harbor

Citation: *Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces*, arXiv:2601.11868; announcement: tbench.ai/news/announcement-2-0. **Artifacts:** dataset `harborframework/terminal-bench-2.0` (HF), harness docs harborframework.com.

What it is

89 curated hard terminal tasks (selected from 229 contributed) across 16 categories (software engineering, security, scientific computing, data science, debugging, games, …) and three difficulty levels. Each task = instruction + Docker environment + automated tests. **Harbor** is the official harness: it runs agents (Terminus 2 — the neutral reference agent — plus Claude Code, Codex CLI, OpenHands, Mini-SWE-Agent) against tasks, e.g.:

```
harbor run -d terminal-bench/terminal-bench-2 -a <agent> -m <model-id> -n <concurrency>
```

Verified so far: install + `harbor run` flags (`-d` dataset, `-a` agent, `-m` model, `--env` remote executor, `-n` concurrency); an `oracle` agent exists for validating setup. **Trajectory/log output location and format are**

not documented on the tutorial page — must be inspected from a real local run before writing the ingest parser (spec 002 requirement).

Relevance to this project

- The task environment for all trajectory collection (pilot: Terminus 2, ~30–50 trajectories, spec 004).
- Task categories and difficulty labels give free stratification variables for later analysis (subquestions 3–4).

4. Synthesis: what our taxonomy takes from each

Design element	Source	Our adaptation
Axis A: cognitive function	AgentErrorTaxonomy's 5 modules	Kept as-is; definitions rewritten for terminal agents
Axis B: fine-grained error type	TRAIL's ~20 types + AgentErrorTaxonomy's 17	Merged, deduplicated, terminal-specific types added
Location annotation	TRAIL (span id) + AgentDebug (step)	Event index/span in normalized trajectory
Impact/severity	TRAIL (Low/Med/High)	Kept
Root-cause vs. cascade	AgentDebug (critical step)	Boolean root-cause flag + cascade links
Empirical revision	(our contribution, subq 1)	Versioned taxonomy with evidence-cited revision log

Sources: TRAIL (arXiv) · TRAIL HTML · Where LLM Agents Fail (arXiv) · AgentDebug repo · Terminal-Bench 2.0 announcement · Harbor docs · Terminal-Bench paper